

Software Security Analysis

Installing the C Compiles, GCC and Analyzing C Program in IDA Pro

Introduction

This lab is designed to develop, compile, and analyze programs using C programming language. The primary goal is to write a program that accepts user input and verifies it against a predefined password using conditional statements. With the comparison result, it executes different functions to indicate whether the password entered is correct or incorrect.

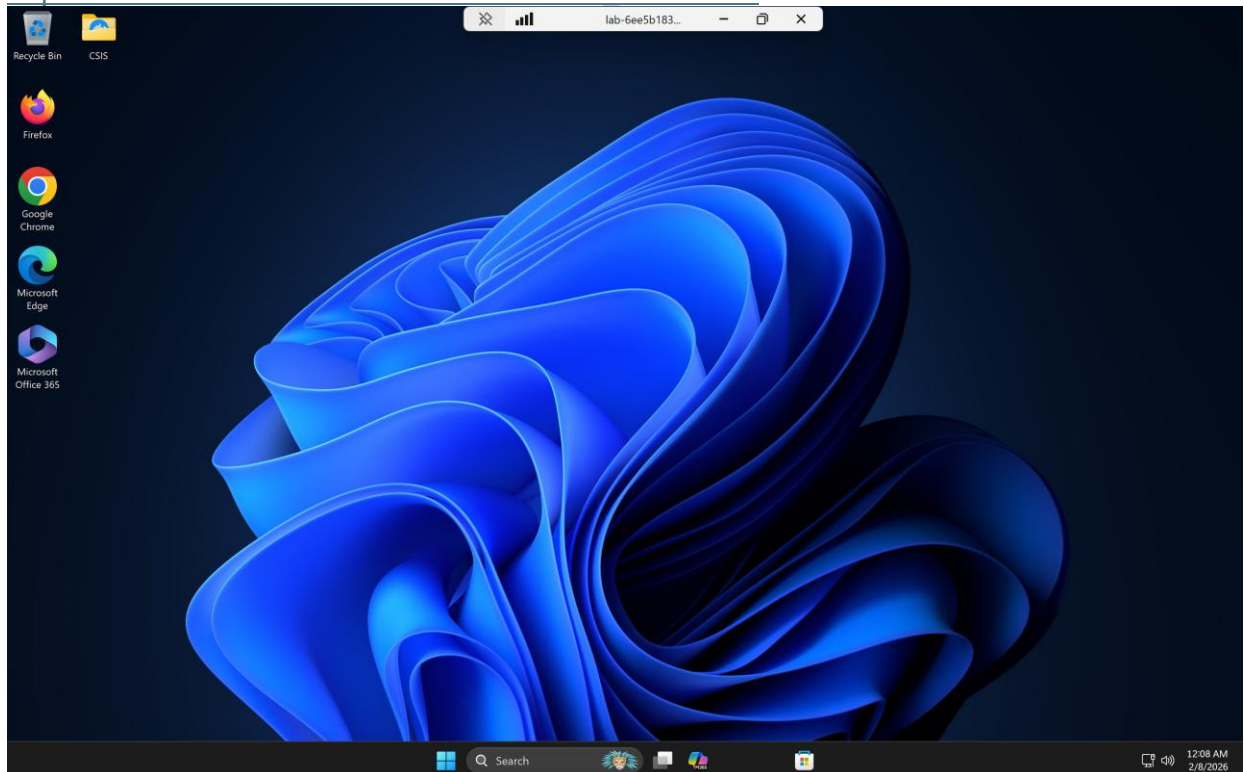
Then the reverse engineering was performed in the IDA Pro using the compiled executable file. The IDA Pro helps to understand how simple programs are converted into machine level operations, including its functions, instructions, and control flow. Used debugging techniques such as setting breakpoints and stepping through code execution to observe the program in real time.

This lab combined programming and analysis of the program to strengthen understanding of both development and reverse engineering processes.

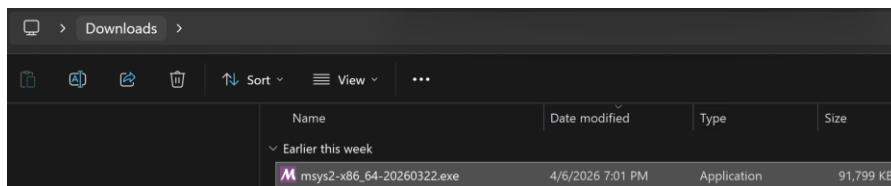
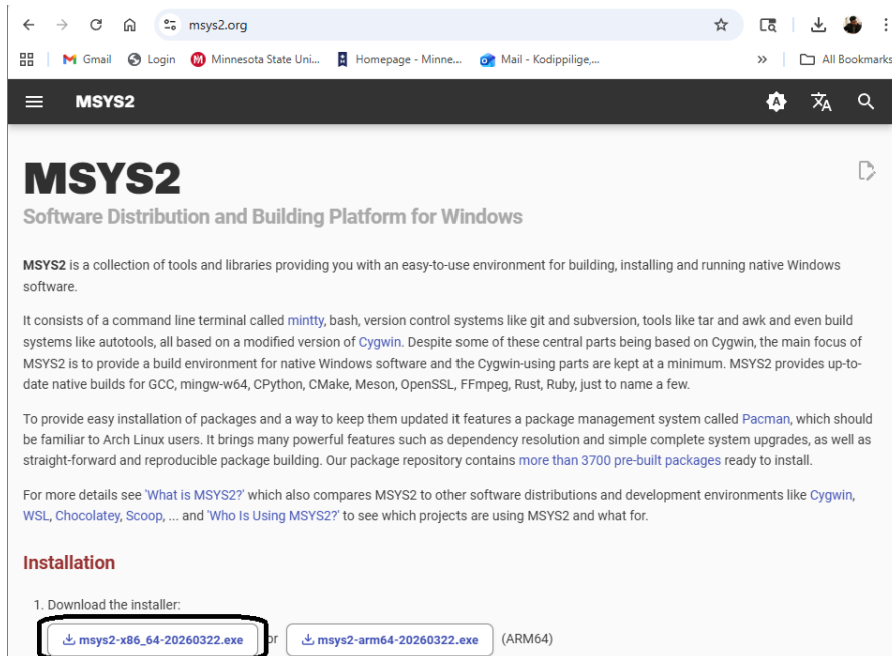
Part 1 - Installing the C Compiles, GCC

1. Logged to Azure Lab environment using following web link and logged into the Windows VM using Oracle VirtualBox.

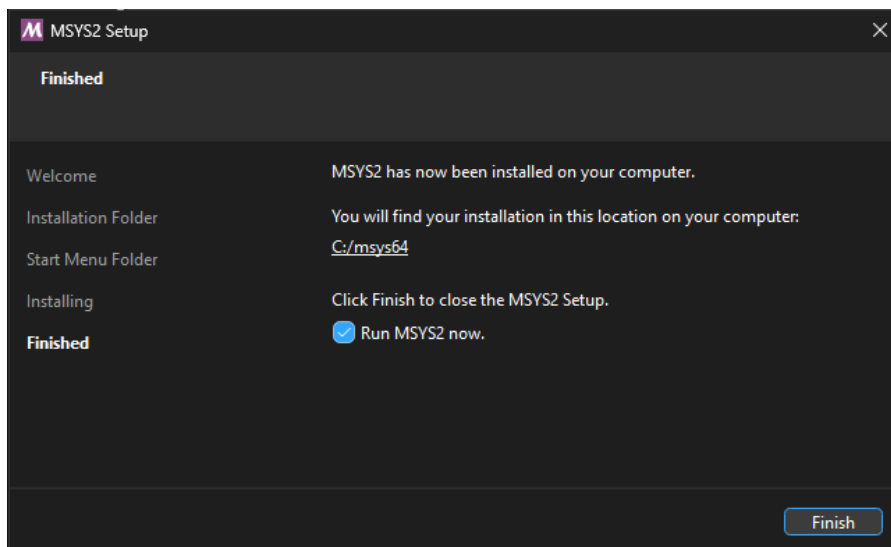
https://labs.azure.com/virtualmachines?feature_vnext=true



2. Downloaded the installer for MSYS2 using URL <https://www.msys2.org/>



3. Installed MSYS2 on Windows VM.



4. Installed the GCC compiler for compiling C programs using `pacman -S mingw-w64-ucrt-x86_64-gcc` command on MSYS2 terminal. Accepted default settings.

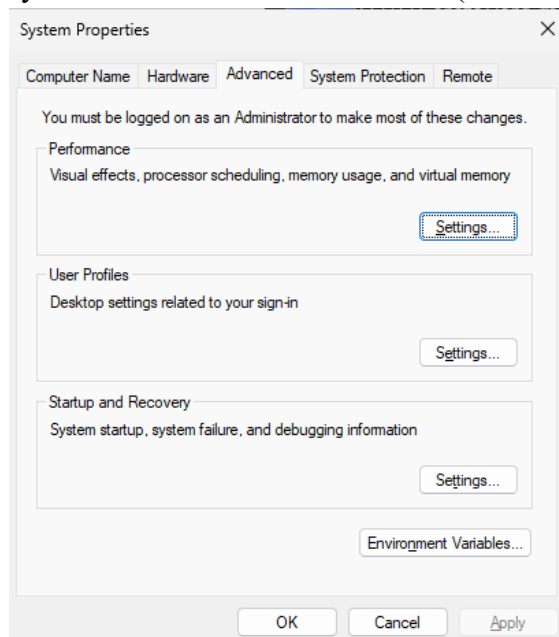
```
CSIS-Admin@lab8K4EB1 UCRT64 ~  
# pacman -S mingw-w64-ucrt-x86_64-gcc  
resolving dependencies...  
looking for conflicting packages...  
  
Packages (17) mingw-w64-ucrt-x86_64-binutils-2.46-2  
mingw-w64-ucrt-x86_64-crt-13.0.0.r560.g3197fc7d6-1  
mingw-w64-ucrt-x86_64-gcc-libs-15.2.0-13 mingw-w64-ucrt-x86_64-gettext-runtime-1.0-1  
mingw-w64-ucrt-x86_64-gmp-6.3.0-2  
mingw-w64-ucrt-x86_64-headers-13.0.0.r560.g3197fc7d6-1  
mingw-w64-ucrt-x86_64-isl-0.27-1 mingw-w64-ucrt-x86_64-libiconv-1.19-1  
mingw-w64-ucrt-x86_64-libwinpthread-13.0.0.r560.g3197fc7d6-1  
mingw-w64-ucrt-x86_64-mpc-1.3.1-2 mingw-w64-ucrt-x86_64-mpfr-4.2.2-1  
mingw-w64-ucrt-x86_64-tzdata-2026a-1  
mingw-w64-ucrt-x86_64-windows-default-manifest-6.4-4  
mingw-w64-ucrt-x86_64-winpthread-13.0.0.r560.g3197fc7d6-1  
mingw-w64-ucrt-x86_64-zlib-1.3.2-2 mingw-w64-ucrt-x86_64-zstd-1.5.7-1  
mingw-w64-ucrt-x86_64-gcc-15.2.0-13  
  
Total Download Size: 69.89 MiB  
Total Installed Size: 550.10 MiB  
  
:: Proceed with installation? [Y/n]
```

5. Verified the installation using `gcc --version` command.

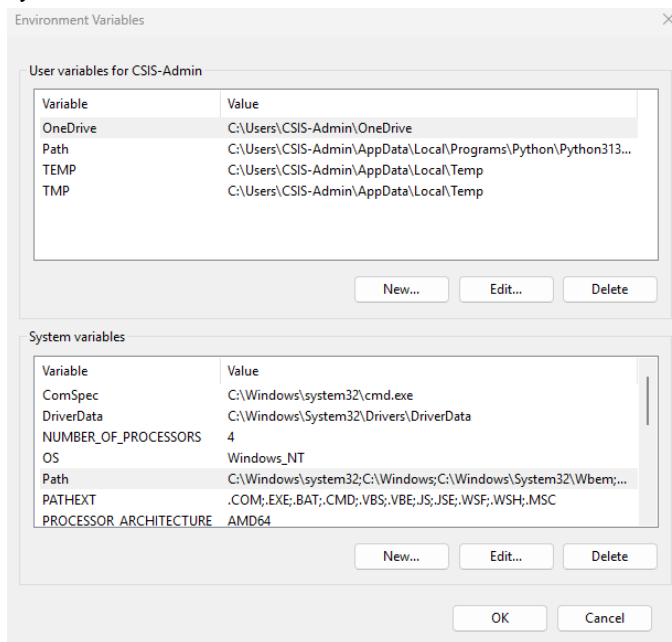
```
CSIS-Admin@lab8K4EB1 UCRT64 ~  
# gcc --version  
gcc.exe (Rev13, Built by MSYS2 project) 15.2.0  
Copyright (C) 2025 Free Software Foundation, Inc.  
This is free software; see the source for copying conditions. There is NO  
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Adding GCC to Path

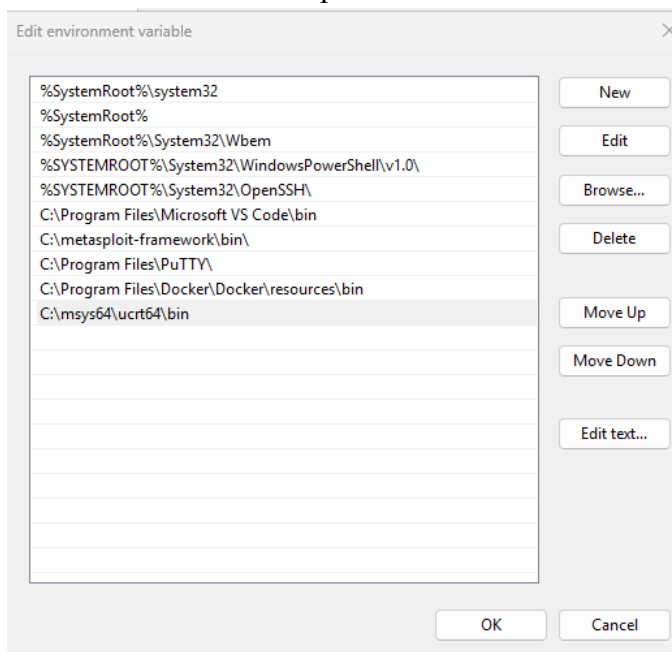
1. Searched 'system environment variables' on Windows search bar and selected Edit the system environmental variables (control panel). It opened the system properties Window.



2. Clicked on Environment Variables button and clicked *Edit* on the *Path* variables under *System variables*.



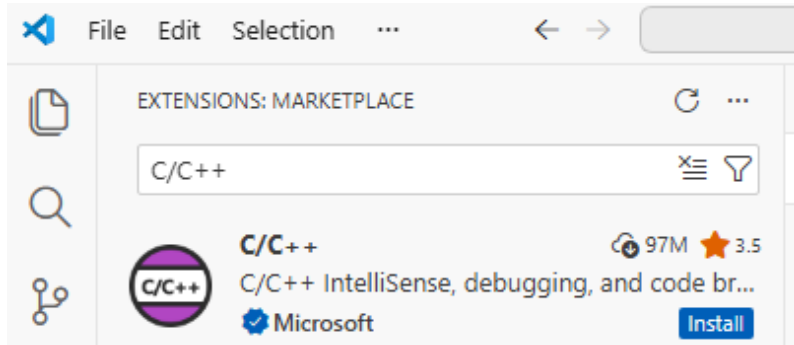
3. Clicked *New* and pasted the full path (C:\msys64\ucrt64\bin) to the GCC bin folder. Then clicked *OK* in all three open Windows.



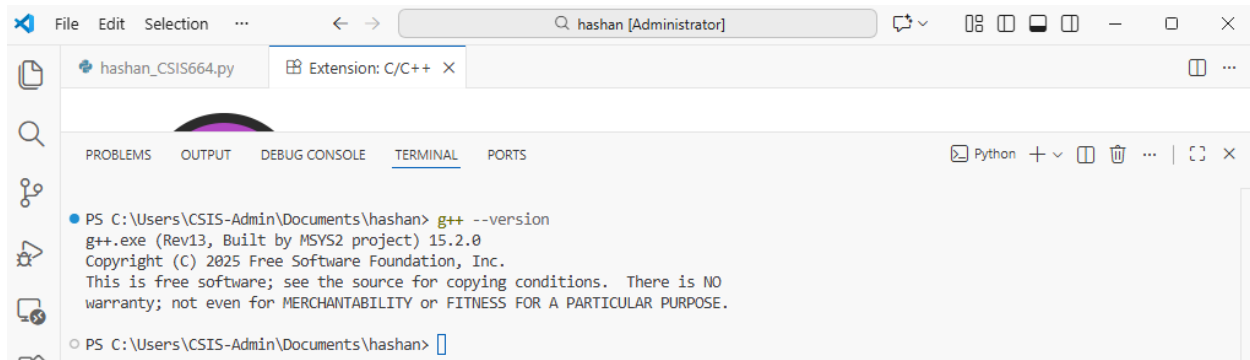
4. closed all open apps.

Part 2 - Preparing VS Code

1. Launched VS Code and installed the C/C++ Extension Pack by selecting *C/C++ IntelliSense, debugging and code browsing*.

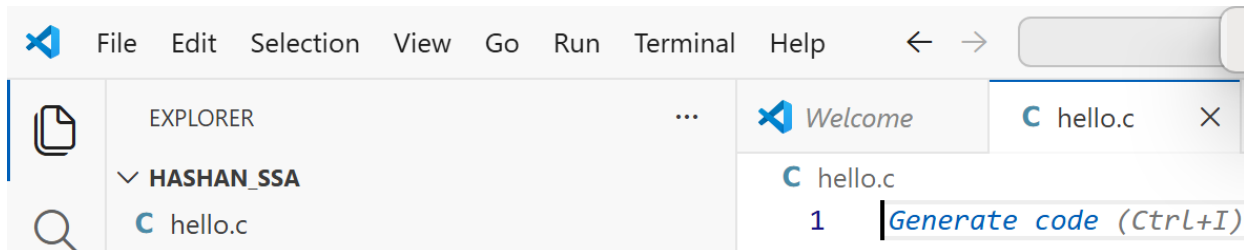


2. Opened a terminal in VS Code and verified that GCC compiler is installed using `g++ --version` command.



Part 3 - Creating and Compiling your C Program in VS Code a Hello World program

1. Launched VS Code and created a folder named *hashan_ssa*. Right-clicked on the folder and created a new C file named *hello.c*

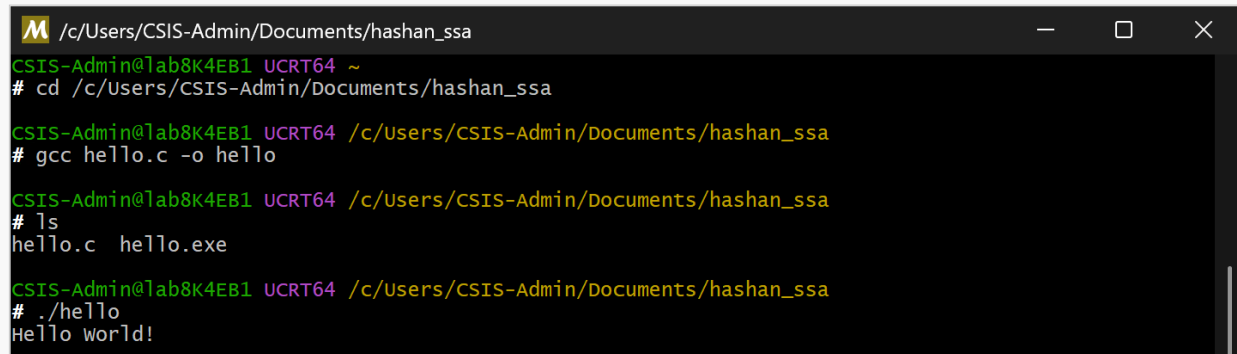


2. Wrote source code for the hello world program in C.



3. Compiled and ran the program to capture the result.

C:\Users\CSIS-Admin\Documents\hashan_ssa>C:\msys64\ucrt64.exe



Part 4 - Creating a program that Authenticates a User

1. Wrote a C program named *hashanAuth.c* by filling following requirements. The C program prompts the user to enter a password and compares it with a predefined password (secret). Based on the comparison, it calls either a function that confirms successful authentication or a function that indicates an incorrect password.

```
C hashanAuth.c X
C hashanAuth.c > main()
1  #include <stdio.h>
2  #include <string.h>
3
4  // Correct password Function
5  void correctPassword() {
6      printf("Hashan entered the correct password\n");
7  }
8
9  // Incorrect password Function
10 void incorrectPassword() {
11     printf("Incorrect password entered\n");
12 }
13
14 int main() {
15     char input[50];
16     char password[] = "secret";
17
18     printf("Enter password: ");
19     scanf("%s", input);
20
21     if (strcmp(input, password) == 0) {
22         correctPassword();
23     } else {
24         incorrectPassword();
25     }
26
27     return 0;
28 }
```

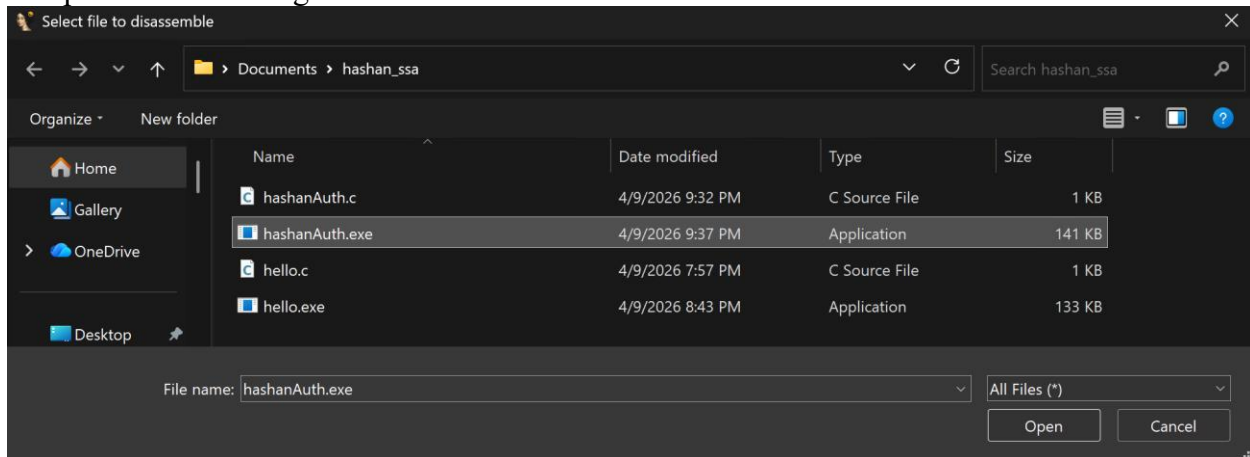
2. Compiled and executed the code to get outputs for different passwords.

```
C:\Users\CSIS-Admin\Documents\hashan_ssa>C:\msys64\ucrt64.exe
```

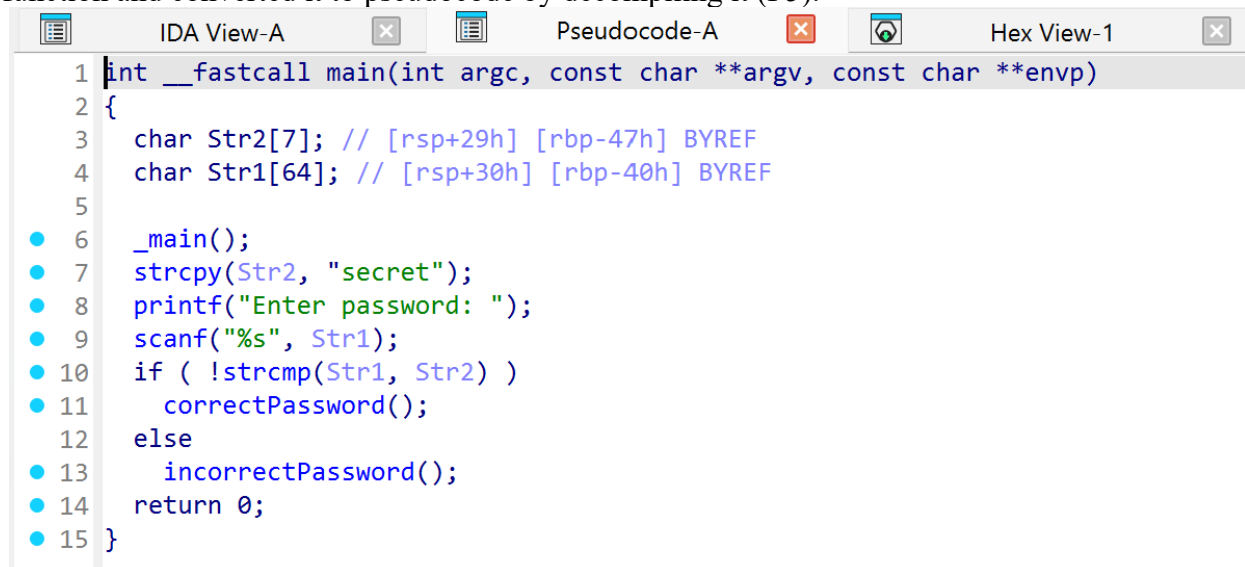
```
M /C:/Users/CSIS-Admin/Documents/hashan_ssa
CSIS-Admin@lab8k4EB1 UCRT64 ~
# cd /C:/Users/CSIS-Admin/Documents/hashan_ssa
CSIS-Admin@lab8k4EB1 UCRT64 /C:/Users/CSIS-Admin/Documents/hashan_ssa
# gcc hashanAuth.c -o hashanAuth
CSIS-Admin@lab8k4EB1 UCRT64 /C:/Users/CSIS-Admin/Documents/hashan_ssa
# ls
hashanAuth.c hashanAuth.exe hello.c hello.exe
CSIS-Admin@lab8k4EB1 UCRT64 /C:/Users/CSIS-Admin/Documents/hashan_ssa
# ./hashanAuth
Enter password: secret
Hashan entered the correct password
CSIS-Admin@lab8k4EB1 UCRT64 /C:/Users/CSIS-Admin/Documents/hashan_ssa
# ./hashanAuth
Enter password: secure
Incorrect password entered
```


Part 5 - Analyzing the Created Program in IDA Pro

1. Opened *hashanAuth.exe* executable file for the program in IDA Pro. Opened as PE file and accepted default settings.



2. Selected each function from my C code from the left side panel. First selected main function and converted it to pseudocode by decompiling it (F5).

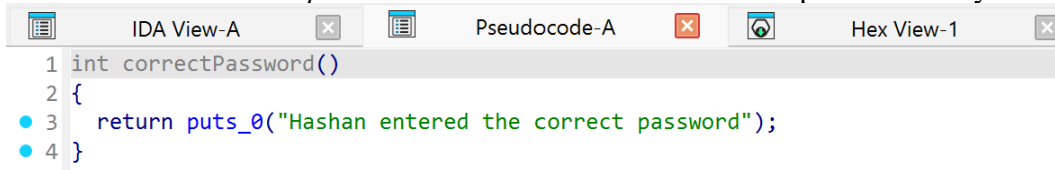


```
1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     char Str2[7]; // [rsp+29h] [rbp-47h] BYREF
4     char Str1[64]; // [rsp+30h] [rbp-40h] BYREF
5
6     _main();
7     strcpy(Str2, "secret");
8     printf("Enter password: ");
9     scanf("%s", Str1);
10    if ( !strcmp(Str1, Str2) )
11        correctPassword();
12    else
13        incorrectPassword();
14    return 0;
15 }
```

The *main* function shows that the program copies the predefined password “secret” into a variable using *strcpy*. Then it uses *scanf* to prompt the user to enter a password and read the input. Then the program compares the password with stored password (secret) and executes different functions with the result of password matching.

The *strcmp* function to compare the user input with the stored password. If the result is equal to zero, the *correctPassword()* function is executed and else the *incorrectPassword()* function is executed.

3. Selected *correctpassword* function and converted it to pseudocode by decompiling it (F5).



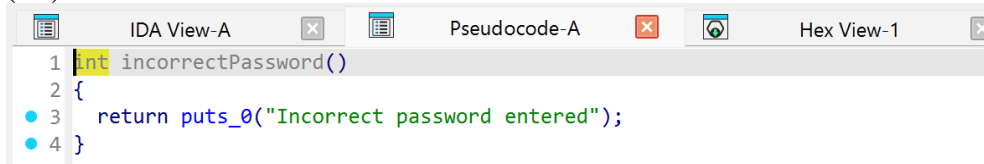
```

1 int correctPassword()
2 {
3     return puts_0("Hashan entered the correct password");
4 }

```

The responsibility of *correctpassword()* function is to display a success message when the correct password is entered. It prints the message *Hashan entered the correct password*.

4. Selected *incorrectpassword* function and converted it to pseudocode by decompiling it (F5).



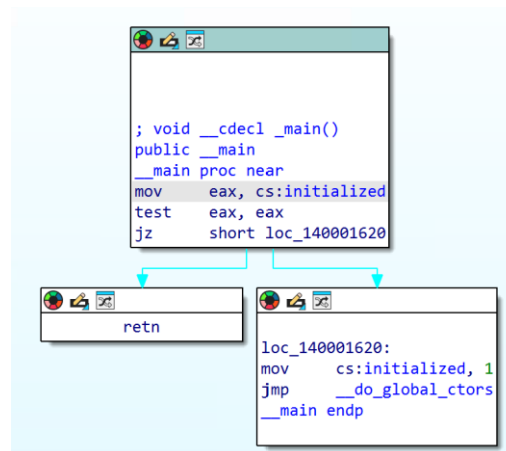
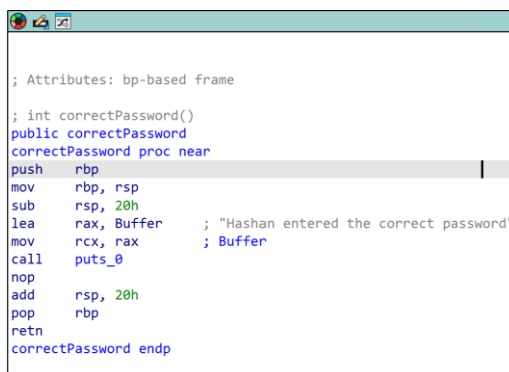
```

1 int incorrectPassword()
2 {
3     return puts_0("Incorrect password entered");
4 }

```

The responsibility of *incorrectpassword()* function is to display a message when the entered password does not match the stored password. It prints the message *Incorrect password entered*.

5. Selected the IDA View-A tab and called functions. It shows the disassembled machine code of the program. Also, it includes low-level instructions such as *mov*, *cmp*, and *call*, which represent how the program operates at the assembly level.

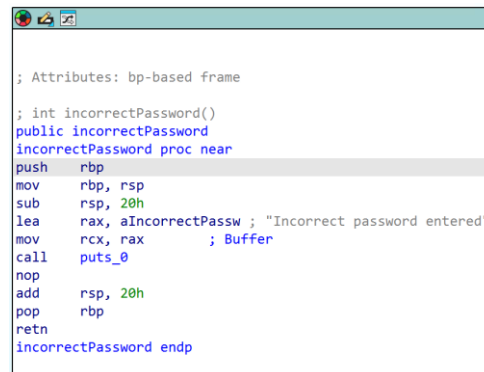



```

; Attributes: bp-based frame

; int correctPassword()
public correctPassword
correctPassword proc near
push    rbp
mov     rbp, rsp
sub     rsp, 20h
lea     rax, Buffer ; "Hashan entered the correct password"
mov     rcx, rax ; Buffer
call    puts_0
nop
add     rsp, 20h
pop     rbp
retn
correctPassword endp

```



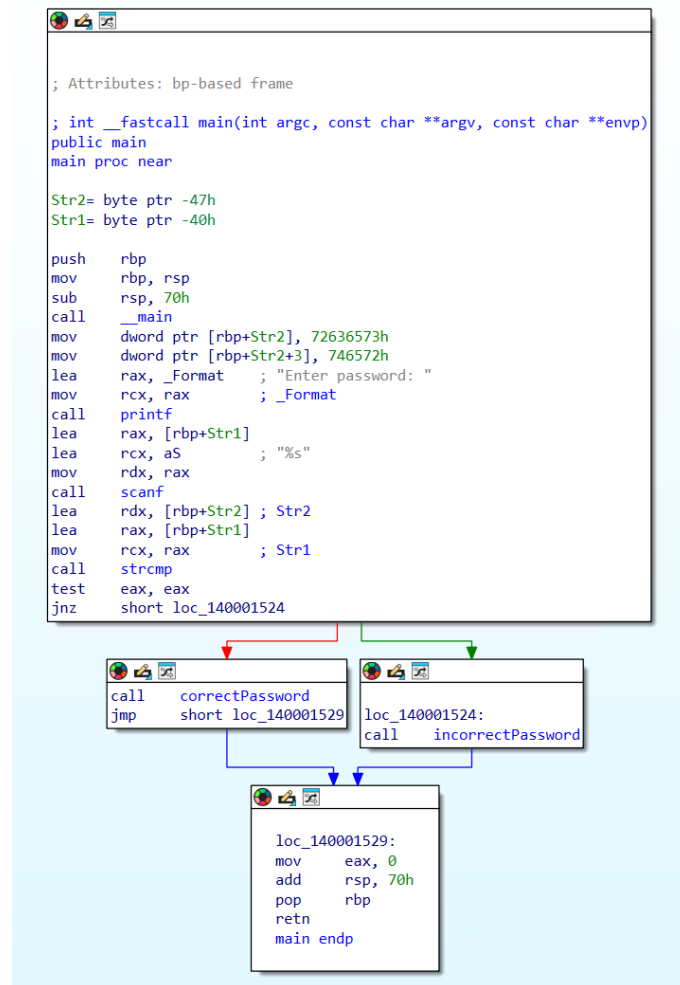
```

; Attributes: bp-based frame

; int incorrectPassword()
public incorrectPassword
incorrectPassword proc near
push    rbp
mov     rbp, rsp
sub     rsp, 20h
lea     rax, aIncorrectPassw ; "Incorrect password entered"
mov     rcx, rax ; Buffer
call    puts_0
nop
add     rsp, 20h
pop     rbp
retn
incorrectPassword endp

```

6. Captured a screenshot of the graph view of *hashanAuth* code disassembly in IDA View-A.

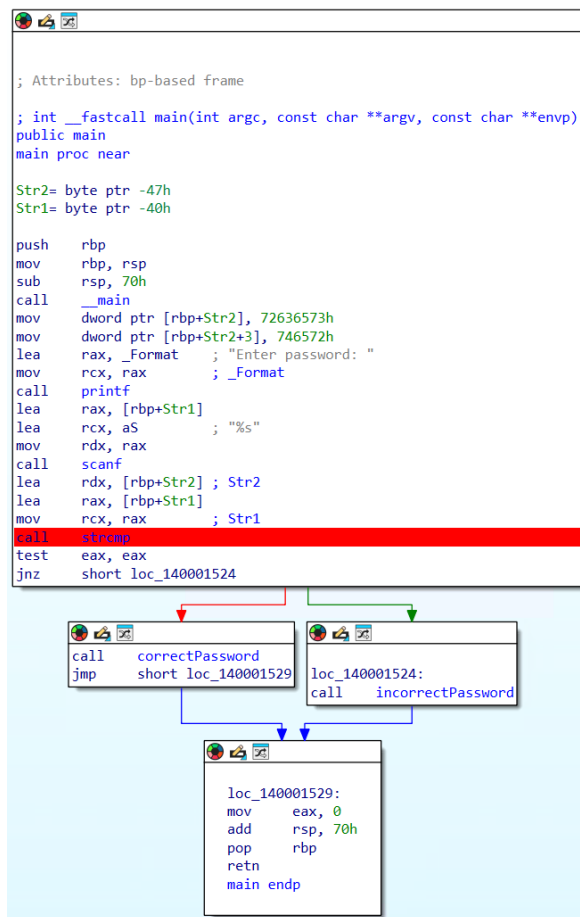


a. Yes, it appears the code is correct looking at the graph. The program uses conditional instruction *test*, and *jnz* after user input is compared with stored password using *strcmp* function. Then the program branches into two paths by showing that the if-else logic in the program is implemented correctly. One leads to the *correctPassword()* function and the other leads to the *incorrectPassword()* function.

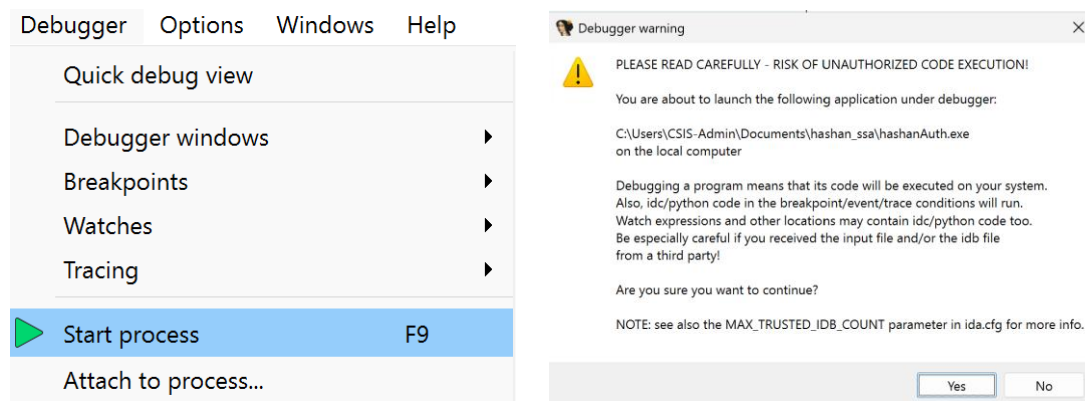
Yes, the branching is correct. The program uses the result of *strcmp* to decide which function to execute. If the password matches, it calls *correctPassword()* function, otherwise it calls *incorrectPassword()* function.

b. The green arrow shows the path when the jump condition is satisfied (*jnz* = true), while the red arrow shows the path when the jump is not taken (*jnz* = false).

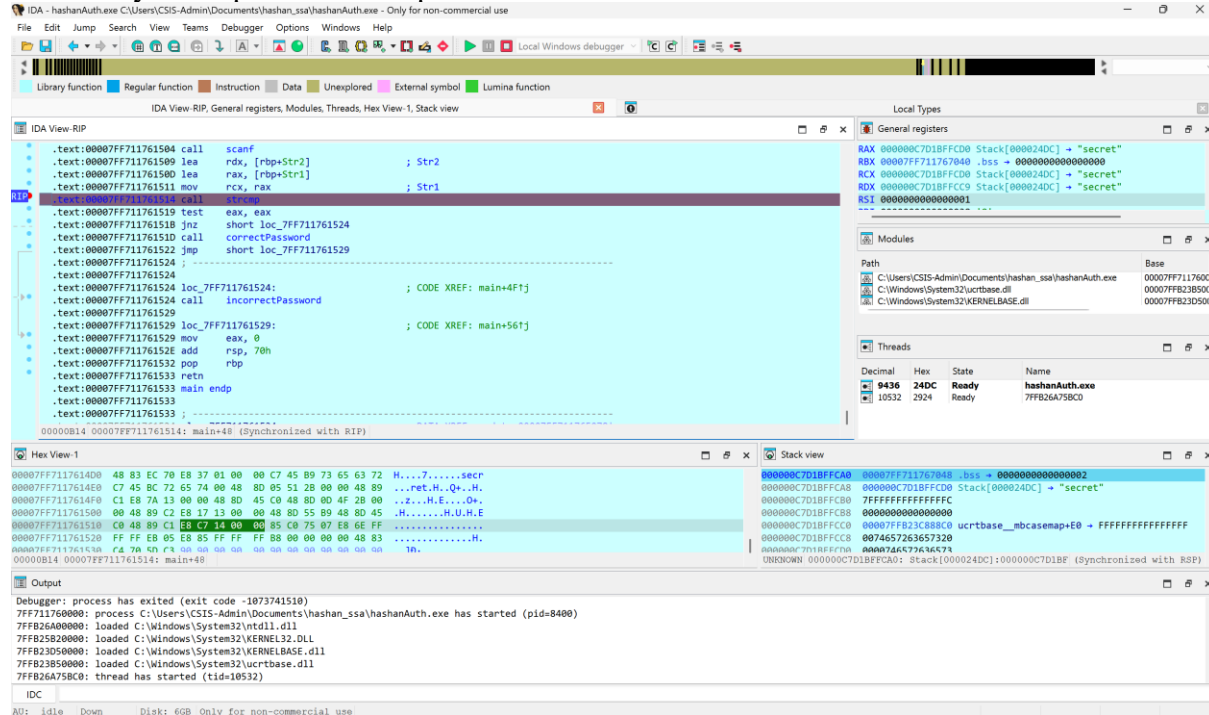
7. Placed a breakpoint on *strcmp* by right-click on the line of code that contains ‘strcmp’ and select Add breakpoint. The *strcmp* function performs the comparison between the user input and the stored password.



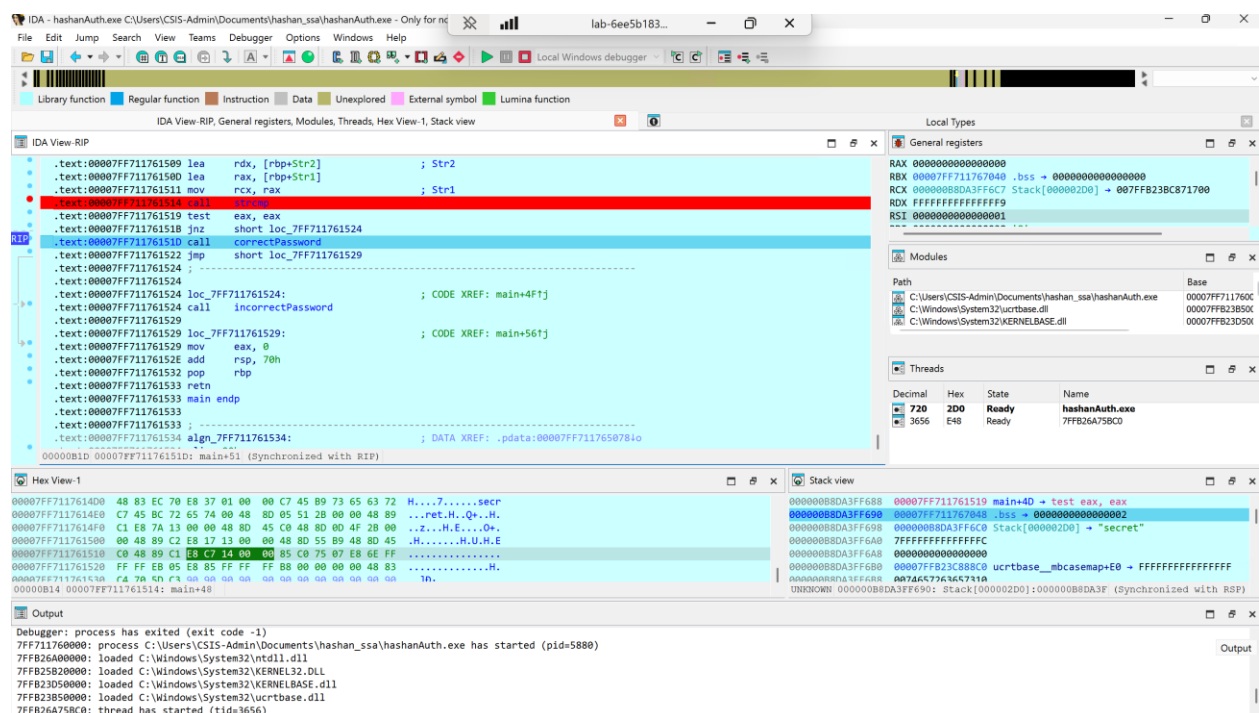
8. Debug the code by using Debugger Menu > Start Process (F9 is also a shortcut). Then accepted “YES” in the warning dialog box.



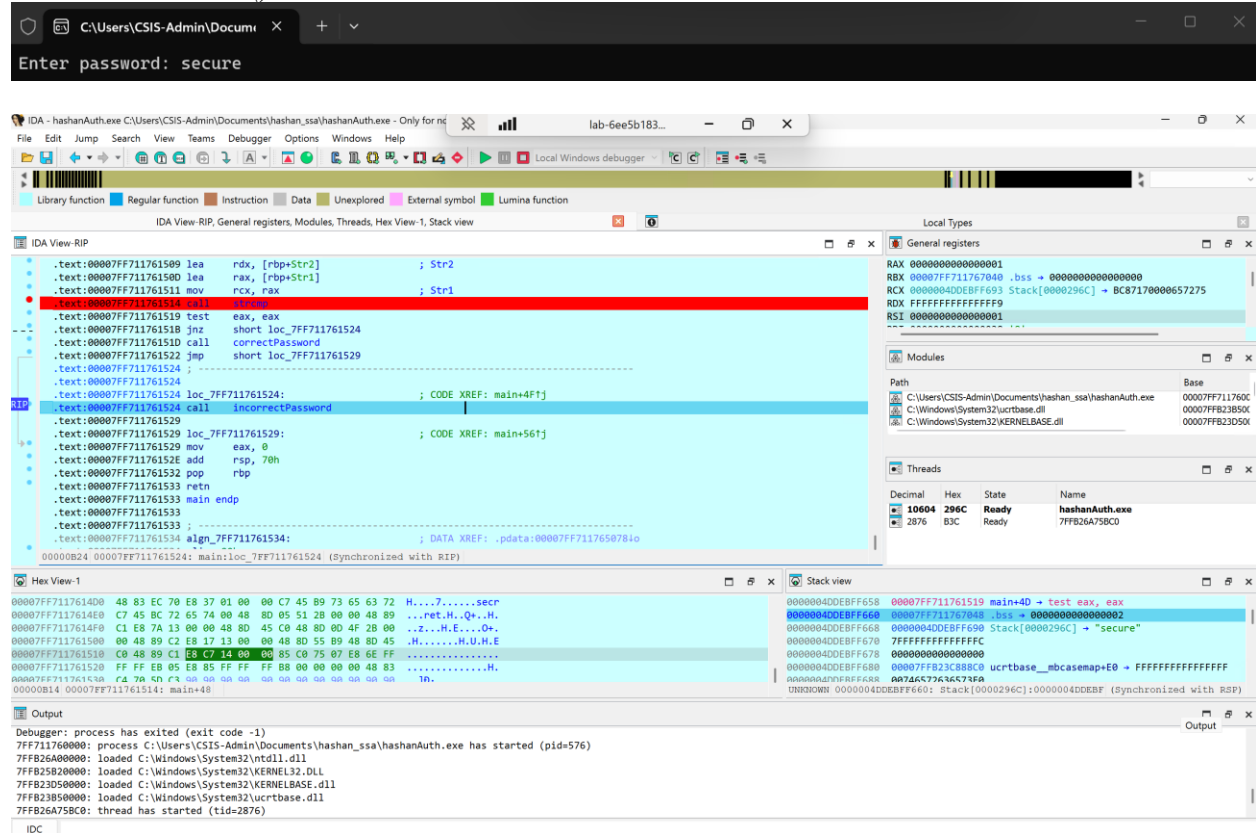
9. The program execution paused at the breakpoint placed on the `strcmp` instruction. It allows us to analyze the password comparison.



10. When I entered the correct password, the execution followed the path leading to the `correctPassword()` function.



11. When I entered a wrong correct password, the execution followed the path leading to the *incorrectPassword()* function.



Used Step Into (F7) to step into the code for different inputs (correct and incorrect passwords) and verify your program is functioning correctly. This confirmed that the authentication logic functions as expected.

Conclusion

This lab helps to develop a simple authentication program in C and tested to verify its functionality. This program compared user input with a predefined password (secure) and executed the appropriate function based on whether the input/password was correct or incorrect. To observe the lower-level function, we can compile and execute C programming languages in a GCC compilation environment.

Analysis performed through IDA Pro can reveal the internal structure of executable files, including imported libraries and assembly-level instructions. Using graph view and disassembly confirmed the branching logic accurately reflects the original if-else structure in the source code. Used debugging tools such as breakpoints and step execution to trace the program's behavior for both correct and incorrect inputs.

This lab covers the practical knowledge of software development, compilation, and reverse engineering. Also, it demonstrated the importance of debugging and reverse engineering techniques in analyzing software behavior.

References

1. OTW. (2023, November 26). *Reverse engineering malware, part 03: IDA Pro introduction*. Hackers Arise. <https://hackers-arise.com/reverse-engineering-malware-part-3-ida-pro-introduction/>
2. Domas, S., & Domas, C. (2024). *x86 software reverse-engineering, cracking, and counter-measures*. Wiley.
3. W3Schools. (n.d.). *C Tutorial*. <https://www.w3schools.com/c/index.php>
4. madsur. (2016, January 30). *IDA Pro newbie tutorial (GBA, SNES, loader script, GDB debugger)*. FF6Hacking. <https://ff6hacking.com/forums/thread-3114.html>
5. Reverse Engineering Stack Exchange. (n.d.). *How to re-analyse a function in IDA Pro?* <https://reverseengineering.stackexchange.com/questions/2309/how-to-re-analyse-a-function-in-ida-pr>